

KI/SZZVP Aplikovaná informatika: volitelné bloky

Objektově orientovaný návrh

1. Zadání úlohy – Architektura herní logiky 2D plošinovky

Vaše nezávislé herní studio vyvíjí prototyp videohry inspirované žánrem plošinových her (např. Super Mario Bros.). Cílem je vytvořit model objektově orientované architektury, která bude řídit chování hlavní postavy, nepřátel, fyzikální interakce (kolize, skoky) a stav herního světa (životy, skóre, časovač).

Vaším úkolem je navrhnout herní systém, který bude:

- spravovat hlavní herní smyčku (Game Loop) zahrnující zpracování vstupů, aktualizaci fyziky, vyhodnocení kolizí a vykreslování,
- evidovat herní entity (Hlavní hrdina, Základní a Obrnění nepřátelé, Statické překážky, Sbíratelné předměty a Power-upy),
- řídit fyzikální stav a modifikátory postavy (Základní, Zvětšený, Nesmrtelný),
- vyhodnocovat kolize a mezipředmětové interakce (sebrání mince, skok na nepřítele, zásah od nepřítele),
- asynchronně komunikovat s vedlejšími systémy (např. Sound/Audio Engine pro přehrávání zvukových efektů).

Povinným výstupem je vypracování 4 klíčových UML diagramů. Termíny v diagramech mohou být v češtině nebo angličtině a mohou se mírně lišit od popisu v sekci 2 (Požadované výstupy). Zmíněné typy náležitostí (např. vztahy či omezující podmínky) by však měly být vždy obsaženy. Nebudeme hodnotit vaše doménové znalosti z oblasti vývoje videoher.

2. Požadované výstupy

2.1 Diagram aktivit – Herní smyčka

Namodelujte algoritmus jednoho snímku/kroku hlavní herní smyčky (Game Loop / Frame Processing):

- Sběr vstupů: načtení akcí od hráče (stisk kláves / gamepadu).
- Aktualizace fyziky a pozic: aplikování gravitace, dopočet rychlostí a nových souřadnic pro všechny aktivní entity v úrovni.
- Vyhodnocení kolizí: detekce překryvů objektů (Hrdina vs. Překážka, Hrdina vs. Nepřítel, Hrdina vs. Mince). Zahrňte rozhodovací uzel na základě toho, zda ke kolizi došlo a o jaký typ objektu jde.
- Aktualizace herního stavu: úprava skóre, odpočet času, kontrola podmínek pro podlehnutí/vítězství (Je hráč stále naživu?, Zbývá hráčovi čas?).
- Paralelní větve: využijte paralelní uzly např. pro oddělení větve pro výpočet vykreslování (Render Engine) a větve pro odbavování zvukových efektů (Audio System).

2.2 Diagram tříd – Datový a objektový návrh

Navrhněte datovou strukturu herního systému s důrazem na využití dědičnosti, polymorfismu a kompozice (např. Level komponuje GameObject) a uveďte kardinality vztahů.:

- Abstraktní třída GameObject (atributy: Pozice X/Y, Rychlost; metody update(), render()),
- Třída PlayerCharacter (dědí z GameObject; atributy: Stav, Životy, Skóre; metody: jump(), takeDamage(), collectPowerUp()),
- Abstraktní třída Enemy (dědí z GameObject; konkrétní třídy WalkerEnemy a ArmoredEnemy dědí z Enemy),
- Třída PowerUp (modifikátory jako GrowthItem, FireAbilityItem),
- Třída Level (obsahuje kolekci GameObject).

Doplňte do diagramu alespoň dvě OCL (Object Constraint Language) omezení:

- „Počet životů hráče nemůže přesáhnout hodnotu 99 a nemůže klesnout pod 0.“
- „Hráč může použít schopnost 'Ohnivá střela' pouze v případě, že jeho aktuální stav je 'EmpoweredState'.“

2.3 Stavový diagram – Chování entity v čase

Namodelujte životní cyklus a transformace Hlavního hrdiny (PlayerCharacter).

- Zachyťte stavy: NormalState, BigState (po sebrání zvětšovacího předmětu), FireState (po sebrání bojového předmětu), InvincibleState (po sebrání ochranné hvězdy) a DeadState.
- Definujte události: onCollectGrowthItem, onCollectFireItem, onCollectInvincibleStar, onTakeDamage, onInvincibleTimeout.
- Uveďte případné hlídací podmínky (např. [lives > 0]).

2.4 Sekvenční diagram – Dynamika konkrétního děje

Vytvořte detailní scénář pro proces "Interakce: Hráč skočí na základního nepřítele (WalkerEnemy)".

- Diagram bude zobrazovat interakci mezi objekty: InputHandler, PlayerCharacter, CollisionEngine, WalkerEnemy, ScoreManager a AudioEngine.
- Zahrňte logiku pro dva případy pomocí alternativního toku:
 - Skok shora na nepřítele: Nepřítel je zničen, hráči se přičte skóre.
 - Náraz do nepřítele z boku: Hráč utrhne poškození / zmenší se.
- Vyjádřete asynchronní volání (např. asynchronní požadavek směrem k AudioEngine pro přehrání zvuku zničení nepřítele bez blokování herní smyčky).

3. Technologické požadavky

- **Modelovací nástroj:** výhradně nástroje pro ruční kreslení (např. diagrams.net / draw.io). Je přísně zakázáno používat generátory z textu (PlantUML, Mermaid).
- **Informační zdroje:** je povoleno využívat internet a generativní AI pro konzultaci konceptů, nikoliv pro generování finálních diagramů.

4. Odevzdání úlohy a obhajoba

Odevzdává se ZIP archiv obsahující:

- Zdrojové soubory diagramů (např. .drawio).
- Exporty v PDF nebo PNG (jasně pojmenované podle typu diagramu).
- README.md:
 - Stručný popis architektury a mechaniky vaší plošinovky.
 - Zdůvodnění návrhu řídicího toku v diagramu aktivit (Game Loop).
 - Zdůvodnění návrhu tříd (hierarchie dědičnosti herních objektů a zapouzdření).
 - Vysvětlení vyhodnocení kolizí a chybových/alternativních stavů v sekvenčním diagramu.

Při obhajobě se zaměřte na:

- Konzistenci mezi diagramy: Zda akce v diagramu aktivit a metody použité v sekvenčním a stavovém diagramu přesně odpovídají rozhraní a metodám definovaným v diagramu tříd.
- Správné použití UML prvků: Správné použití uzlů Fork/Join v aktivním diagramu a vyjádření asynchronní komunikace v sekvenčním diagramu.